

高级语言程序设计

实验报告

南开大学 计算机大类

姓名 方豫馨

学号 2513000

班级 3137

2026 年 5 月 9 日

目录

高级语言程序设计大作业实验报告	1
一、作业题目	1
二、开发软件	1
三、课题要求	1
四. 主要流程	1
1. 整体流程	1
(1) 实现思路:	1
(2) 项目结构与类设计:	3
2.主要算法或公式	3
(1) 玩家移动与方向判定	3
(3) 攻击判定矩形	4
(4) 碰撞检测、伤害和僵直	5
(5) 敌人 AI 状态选择	5
(2)加深了对游戏循环的理解	7
(3)更好地理解资源管理	7
七. AI 工具使用说明	7

高级语言程序设计大作业实验报告

一、作业题目

猫咪大作战

二、开发软件

QT Creator 19.0.0(Community)

三、课题要求

- 1) 面向对象。
- 2) 单元测试。
- 3) 模型部分
- 4) 验证

四. 主要流程

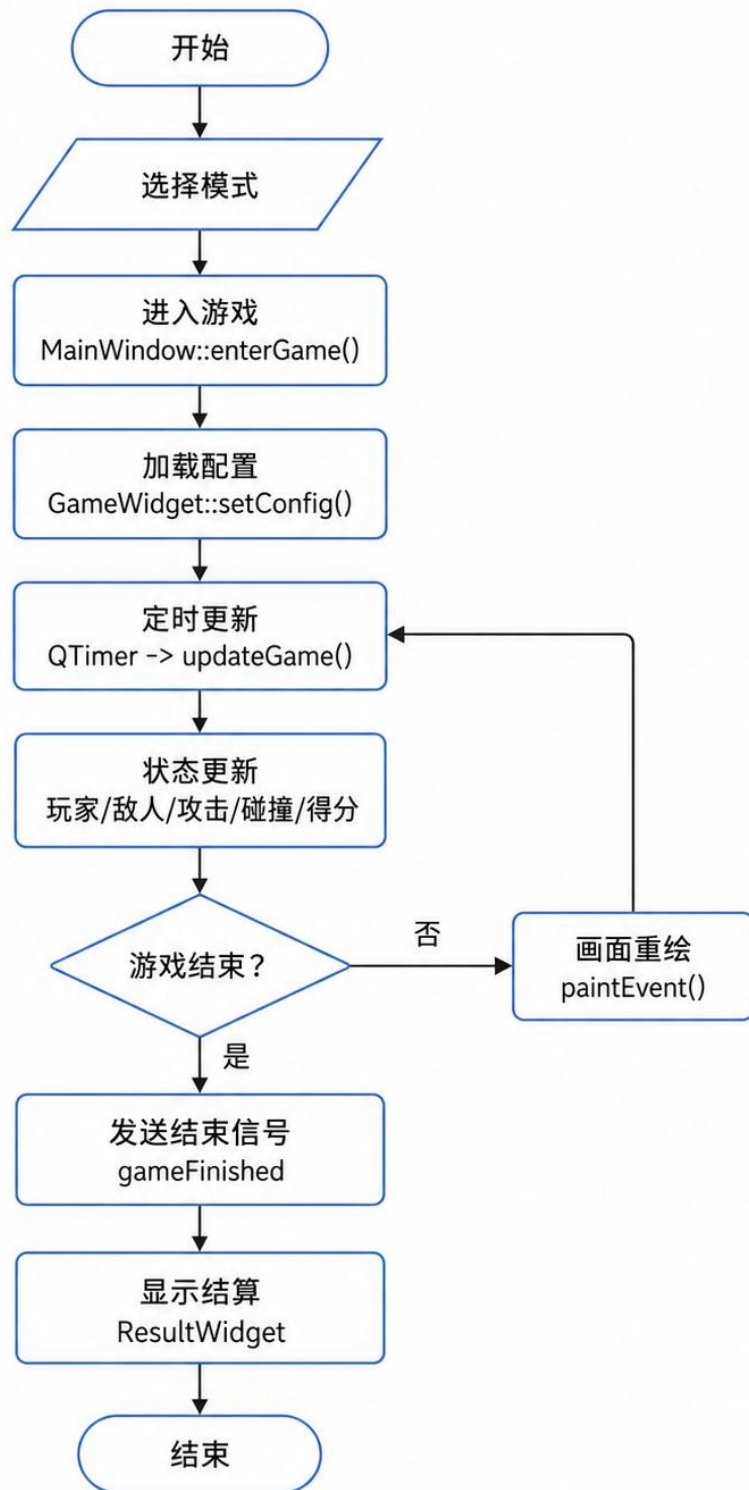
1. 整体流程

(1) 实现思路:

程序启动后进入 MainWindow。MainWindow 内部使用 QStackedWidget 管理三个主要界面：开始菜单 StartMenu、游戏界面 GameWidget、结算界面 ResultWidget。用户在开始菜单中选择关卡模式或无尽模式后，MainWindow 创建并传入 GameConfig，GameWidget 根据配置初始化游戏数据，然后进入由 QTimer 驱动的循环。

GameWidget 的主循环函数为 updategame()。该函数每 20 ms 被触发一次，依次完成玩家冷却更新、玩家移动、回血道具检测、敌人 AI 与移动、攻击框生命周期更新、碰撞结算、死亡敌人移除、胜负判断、无尽模式补怪和画面刷新。

该流程中，输入事件只改变玩家状态或触发技能，真正的位置变化、AI 决策、碰撞和胜负判定统一在定时器循环中处理，体现了游戏循环结构实现思路。流程图如下



(2) 项目结构与类设计:

项目按照 core、entity、combat、ui 和 assets 进行分层。core 管理窗口和游戏主循环，entity 管理可绘制实体与角色，combat 处理技能与伤害判定，ui 处理菜单、HUD、结算等界面，assets 保存图片、音频和特殊彩蛋素材。

文件夹	主要文件	职责
src/core	gamewidget、mainwindow、gameconfig、gametypes	窗口切换、游戏循环、模式配置、枚举类型定义
src/entity	entity、character、player、enemy	实体基础属性、角色动画、玩家输入、敌人 AI
src/combat	skill、attackbox、collision	攻击框创建、持续时间、命中检测、伤害和僵直
src/ui	hud、startmenu、resultWidget	血条、分数、技能图标、开始菜单、结算界面
assets	background、player、enemy、ui、sound、caidan	背景、精灵动画、图标、音效、彩蛋资源

类之间的关系如下：Entity 是最基础的可绘制对象，保存坐标、尺寸、图片和存活状态；Character 继承 Entity，增加生命值、移动速度、攻击力、方向、动作、动画帧、技能冷却、哈气层数、动作锁和僵直时间；Player 与 Enemy 继承 Character，分别实现玩家输入逻辑和敌人 AI 逻辑。Skill 不是实体，而是根据角色方向创建 Attackbox；Collision 负责遍历 Attackbox 并对目标角色造成伤害。

2.主要算法或公式

(1) 玩家移动与方向判定

玩家通过 W、A、S、D 四个按键控制移动。Player::updategame() 根据按键状态得到 dx、dy，并对斜向移动进行归一化处理：

$$len = \sqrt{dx \times dx + dy \times dy}$$

$$dx = \frac{dx}{len}$$

$$dy = \frac{dy}{len}$$

$$epos.x = epos.x + dx \times speed$$

$$epos.y = epos.y + dy \times speed$$

归一化的作用是保证斜向移动速度与水平或竖直移动速度一致。如果不做归一化，斜向同时按下两个方向键时，实际速度会变为 `speed` 的 $\sqrt{2}$ 倍。

（2）动作锁与动画帧更新

Character 使用 Action 表示当前动作，包括 `still`、`run`、`attack`、`ha`、`plusattack`；使用 Direction 表示角色朝向，包括 `up`、`down`、`left`、`right`。动画资源存储在 `sprites[ACTION][DIRECT]` 中，每个动作和方向对应一个 QPixmap 序列。

当角色释放技能时，`playaction()` 会设置当前动作并给予一定 `actionLock`。动作锁期间普通移动动作不能覆盖攻击或哈气动作，从而避免攻击动画刚播放就被行走动画打断。`updateanimation()` 根据 `frameinterval` 控制帧切换，并在动作或方向变化时将 `frameid` 重置为 0。

（3）攻击判定矩形

`Skill::createAttbox()` 根据角色当前方向创建攻击判定矩形。矩形以角色身体中心点为基准，向角色面朝方向延伸 `range` 距离，并以 `thickness` 作为判定宽度。该设计使攻击框始终贴合角色前方，而不是固定出现在地图某一位置。

向右攻击: `QRectF(c.x(), c.y() - thickness / 2, range, thickness)`

向左攻击: `QRectF(c.x() - range, c.y() - thickness / 2, range, thickness)`

向上攻击: `QRectF(c.x() - thickness / 2, c.y() - range, thickness, range)`

向下攻击: `QRectF(c.x() - thickness / 2, c.y(), thickness, range)`

Attackbox 内部还保存 `duration` 和 `hashit`。`duration` 每帧减少，减到 0 后攻击框过期；`hashit` 用于保证同一个攻击框不会在持续时间内对同一个目标反复造成伤害。

(4) 碰撞检测、伤害和僵直

`Collision::handleColl()` 遍历所有未过期攻击框。如果攻击框来自玩家，则检测它是否与敌人矩形相交；如果来自敌人，则检测它是否与玩家矩形相交。命中后调用 `hurt()` 扣除生命值，再调用 `addStiff()` 添加僵直时间，并将攻击框标记为已命中。

僵直的意义是让受击对象在短时间内不能继续移动或释放动作，使打击反馈更加明确。当前实现中哈气的僵直时间更长，因此它不仅是叠层技能，也具有一定控制效果。

(5) 敌人 AI 状态选择

`Enemy::updategame()` 根据玩家位置计算距离和方向。敌人具有视野范围 `sightrange`、攻击范围 `attackrange`、哈气范围 `harange`，以及 `prepareTimer` 预备时间。敌人进入攻击范围后不会立刻攻击，而是需要达到 `prepareNeed`，避免玩家一接触敌人就被立即攻击。

`Enemy::chooseState()` 根据敌人类型、技能冷却、玩家距离和随机数选择 `aiState`。纸箱怪只会移动和普通攻击；橘猫怪可以普通攻击、哈气，并在哈气层数达到 3 后释放强化攻击。这样两个敌人虽然共享 `Character` 和 `Enemy` 的基础框架，但在技能组合和行为表现上有所区别。

3. 单元测试

不同窗口是否能正确切换，玩家与敌人状态是否按规则更新，攻击与碰撞是否只结算一次，胜负和分数是否正确，资源、音效和暂停功能是否稳定等。

五. 单元测试

测试内容如下

编号	测试内容	预期结果	结果
1	启动程序，点击关卡模式	进入游戏界面，背景、玩家、暂停按钮、HUD 正常显示	通过
2	按 W/A/S/D 移动并靠近边界	角色按方向移动，斜向移动不加速，无法离开地图	通过

3	玩家左键普攻命中敌人	生成玩家攻击框，敌人扣 10 血，攻击框标记已命中	通过
4	连续按 E 释放哈气	每次哈气增加层数，达到 3 层后强化攻击准备完成	通过
5	三层哈气后按 Q	释放强化攻击，造成 25 伤害，halayer 清零，plusready 变 false	通过
6	敌人进入攻击范围	敌人先预备，再根据类型释放攻击；纸箱怪只普攻，橘猫怪可哈气和强化攻击	通过
7	玩家被敌人攻击命中	玩家扣血并进入短暂僵直，僵直期间不能移动	通过
8	击败橘猫怪和纸箱怪	橘猫怪加 20 分，纸箱怪加 10 分，死亡敌人被移除	通过
9	关卡模式清空全部敌人	触发胜利结算并停止背景音乐和战斗音效	通过
10	玩家生命值归零	触发失败结算，显示当前分数	通过
11	无尽模式中击败敌人	当场上敌人数低于 config.enemynum 时自动补怪	通过
12	玩家拾取回血道具	生命值增加 10，但不超过 maxhp	通过
13	点击暂停按钮	显示暂停遮罩；继续游戏后恢复；回到菜单停止计时器	通过
14	切换彩蛋模式后进入游戏	玩家显示和部分音效切换为彩蛋资源	通过

测试过程中重点检查了以下易出错问题：第一，攻击框持续多帧时不能重复扣血，因此 Attackbox 增加 hashit 并在命中后 markHit()；第二，敌人不应在接触玩家瞬间立刻攻击，因此 Enemy 增加 prepareTimer；第三，角色图片较大时完整矩形碰撞会显得过于僵硬，因此移动阻挡使用缩小后的 bodyRect。

六. 收获

(1)加深了对面向对象设计的理解

在制作这个项目的过程中，我最大的收获是对继承和职责划分有了更具体的理解。最初如果我把所有函数全部写在 `GameWidget` 中，代码会变得难以维护。拆分后，`Entity` 只关心基础坐标和绘制，`Character` 管理角色通用属性和动画，`Player` 管理玩家，`Enemy` 管理敌人，`Skill` 和 `Collision` 管理战斗判定，`GameWidget` 只负责调度。这样的结构更符合面向对象程序设的特点。

(2)加深了对游戏循环的理解

我明白了 Qt 程序不是普通的顺序执行程序，而是通过事件和信号槽响应用户操作。游戏中的按键、鼠标点击、按钮点击、定时器 `timeout`、界面切换都属于事件驱动。通过 `QTimer` 固定周期调用 `updategame()` 后，程序形成了稳定的游戏循环，输入事件只改变状态，循环负责统一结算。

(3)更好地理解资源管理

在项目开发中，我一开始将所有文件放在 `resource.qrc`，导致文件增多后堆空间不足的问题。后来经过 AI 指导，我学习了通过 `ASSET(path)` 宏、`assetRoot()` 搜索 `assets` 文件夹的方法来解决，这也使素材替换和打包更方便。

七. AI 工具使用说明

工具名称及版本：ChatGPT（GPT-5.5 Thinking）；OpenAI 图像生成工具。

具体用途：生成游戏素材、辅助分析项目文件结构、辅助分析代码运行问题，项目报告润色。

使用位置：项目所需的图片素材、指导项目文件结构划分，部分功能提示分析（相对路径，碰撞重叠的避免，彩蛋的管理）